Image Classification with ResNet as Backbone

112550097 Hung-I, Yang

March 30, 2026

1 Introduction

1.1 Task Description

The dataset contains around 23,000 color images across 100 classes. The standard split provides 21,024 training images and 23,44 test images. Each image belongs to exactly one category. The objective of this image classification task is to train a model with ResNet [1] as backbone that predicts the correct class label among the 100 options. I uploaded the training code here: https://github.com/harry0816web/NYCU-VRDL/tree/main/hw1.

1.2 Core Idea

My final approach uses three key components to maximize generalization and accuracy: **1.Transfer Learning** with a pre-trained **ResNeXt-50** [2] backbone, **2.Data augmentation** to reduce overfitting and improve robustness, and **3.Bagging** across multiple random seeds to stabilize optimization and improve final performance.

2 Method

2.1 Data Preprocessing & Augmentation

The training transform pipeline applies:

- **Input resizing**: Due to the size variation of the dataset, we first apply `RandomShortestSize` to sample the short-side length in $[256, 480]$ and then crop to **224×224** using `RandomCrop`. This produces consistent inputs for the 224x224 backbone while still providing scale diversity during training.
- **RandomHorizontalFlip**: add geometric robustness.
- **Color jitter**: apply color jittering to the image to improve robustness.
- **TrivialAugmentWide** [3]: uniformly sample a transformation technique and strength then apply to the image, simple but very effective.
- **Normalize**: align input statistics with the pre-trained backbone.
- **Random Erasing** [4]: randomly mask some regions of the image to prevent model overly rely on specific patterns.

Together, these operations broaden the training distribution, reduce reliance on spurious patterns, and mitigate overfitting.

2.2 Model Architecture

2.2.1 Backbone (ResNeXt-50)

I use the `resnext50_32x4d` backbone initialized with **IMAGENET1K_V2** pre-trained weights [5]. Compared with the original ResNet [1] architecture, which relies on a single sequential path within its bottleneck blocks, ResNeXt introduces a highly modularized multi-branch architecture based on a "split-transform-merge" strategy. Inspired by Inception models but avoiding their heavily customized and complex branch designs, ResNeXt standardizes the network by ensuring all parallel paths share the exact same topology.

To conceptualize this simplicity, the authors propose a "Network-in-Neuron" hypothesis. A basic fully connected neuron fundamentally performs a "split-transform-merge" operation: it splits the input vector $\mathbf{x}$ into single-dimensional components $(x_1, x_2, \ldots)$, transforms each component by scaling it with a specific weight $(w_i x_i)$, and finally merges the results by summing them up $(\sum_i w_i x_i)$. ResNeXt scales up this fundamental operation by replacing the simple weight transformation with a miniature neural network (e.g., a bottleneck layer of $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ convolutions). This upgrade creates a remarkably powerful "Aggregated Transformations" structure while maintaining strict and simple design rules.

This design exposes a new, essential architectural dimension called **cardinality**, which is the size of the set of transformations. In practice, this multi-branch aggregation is elegantly and efficiently realized through grouped convolutions. The network splits the input channels into multiple lower-dimensional subspaces, transforms them independently through 3×3 filters, and merges them by concatenation before a final 1×1 convolution aggregation and residual addition.

All in all, by reallocating parameters from a single wide convolution into multiple narrow, independent paths, the aggregated transformations achieve stronger feature fitting capabilities. Empirical evidence from the paper shows that increasing cardinality not only lowers the testing error but also significantly reduces the training error compared to a standard ResNet under the strict constraint of maintaining the same computational complexity and parameter count. And that's why I choose ResNeXt-50 as the backbone.

2.2.2 Final Layer (MLP / Classifier Head)

Since the dataset has 100 classes (unlike ImageNet's 1000 classes), I replace the original final fully connected layer with a custom classifier head that outputs 100 logits with dropout to prevent overfitting.

2.2.3 Params Count

The ResNeXt-50 backbone has 25 million parameters, as I change the last fully connected layer to output 100 logits, the total number of parameters is 25 million - 2 million + 0.2 million= approximately 23.2 million.

2.3 Hyperparameter Settings & Training Strategy

Started with the default settings from the original ResNet paper [1], I visualize training curves with W&B package, then analysis and optimize setting based on the results of each experiment. This is the final training settings:

- **Optimizer:** AdamW
- **Scheduler:** CosineAnnealingLR

- **Separate Learning Rate for Backbone and FC:**
  - Backbone LR: 10^{-4}, use a smaller learning rate for the backbone to prevent it from forgetting the pre-trained features.
  - FC LR: 10^{-3}, use larger learning rate for the new FC layer.
  - Weight Decay: $5e^{-4}$
  - Dropout: 0.5
  - Label Smoothing: 0.1, soften the targets to prevent overfitting.
  - Batch Size: 64
  - Epochs: 100

3 Results

3.1 Training & Validation Curves

3.1.1 Choose Backbone

I use same training settings for both ResNet-50 and ResNeXt-50 to compare their performance. As can be seen from the figure 1, ResNeXt-50 achieves better performance on both training and validation sets. Furthermore, the test score table 1 shows that ResNeXt-50 achieves a higher test score than ResNet-50.

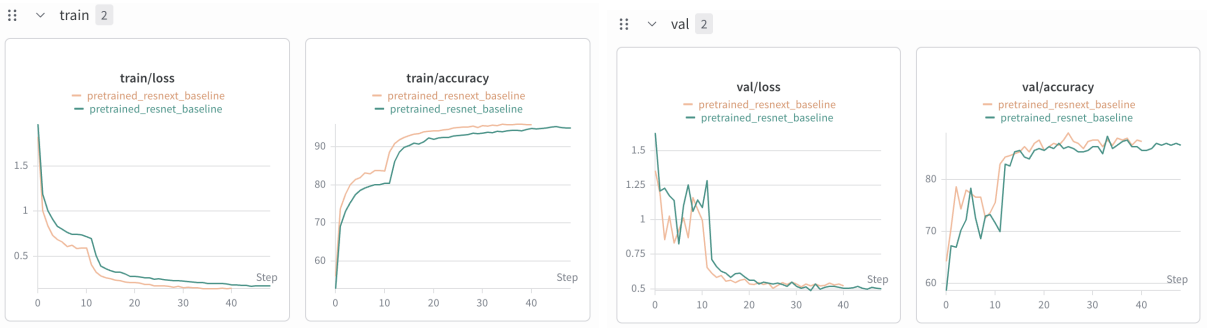

Figure 1: Training and validation loss curves of ResNet-50 and ResNeXt-50.

Table 1: ResNet-50 vs. ResNeXt-50 performance

Backbone	Test Score
ResNet-50	0.92
ResNeXt-50	0.93

3.1.2 Trivial Augmentation

I add some regularization techniques, such as label smoothing, weight decay, etc. to prevent overfitting. But as can be seen from the figure 1, while training loss is still decreasing and finally close to 0, the validation loss stop decreasing at about 60 epochs. Which means the model is still overfitting, therfore I add trivial augmentation [3] to improve the generalization ability of the

model. After adding trivial augmentation, we can see that the validation loss is decreasing, and get a better validation accuracy. What's more, the test score table 2 shows that the test score of the model with trivial augmentation is higher.

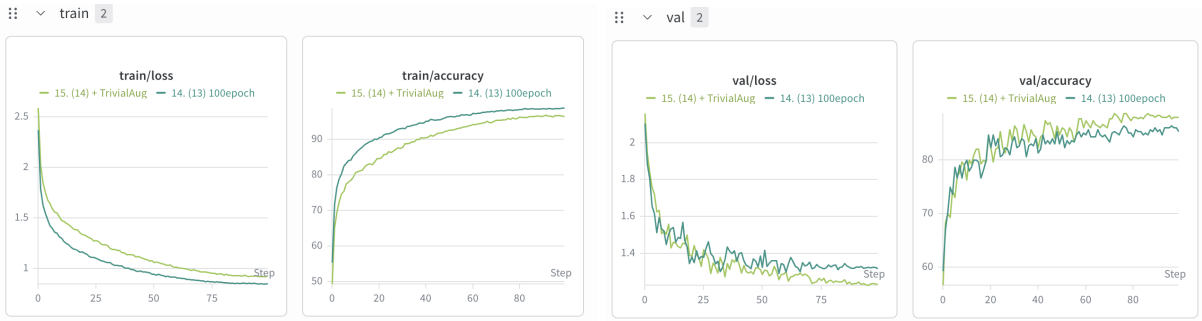

Figure 2: Training and validation loss curves of ResNeXt-50 w/ and wo/ trivial augmentation.

Table 2: w/ trivial augmentation vs. wo/ trivial augmentation performance

Method	Test Score
w/ trivial augmentation	0.94
wo/ trivial augmentation	0.93

3.1.3 Separate Learning Rate & Random Erasing

I use separate learning rate for the backbone and the final classifier head for the pretrained-weight to preserve its capability to capture features, and figure 3 shows some interesting results. Model with seperate learning rate achieves highe validation accuracy at around only 20 epochs, which is also higher than the highest validation accuracy of the model without seperate learning rate. I think the reason is that the pretrained-weight is already trained on a large dataset, so it has already learned a lot of features, what we need to do is just learn the new classifier head to fit the new classes, so that it can achieve higher validation accuracy at such few epochs. However, this also means that valildation loss stopped to decrease at early stage while training loss is still decreasing, which means the model is still overfitting. Therefore, I further add random erasing [4] to the training set to improve the generalization ability of the model. After adding random erasing, we can see that the validation loss is decreasing, and get a better validation accuracy. However, both separate learning rate and random erasing can't improve the test score. (Table 3)

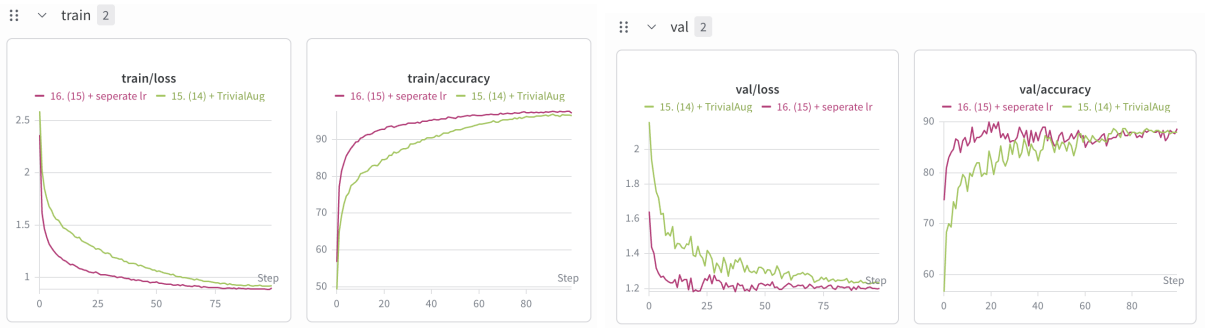

Figure 3: Training and validation loss curves of model from 3.1.2 w/ and wo/ seperate learning rate.

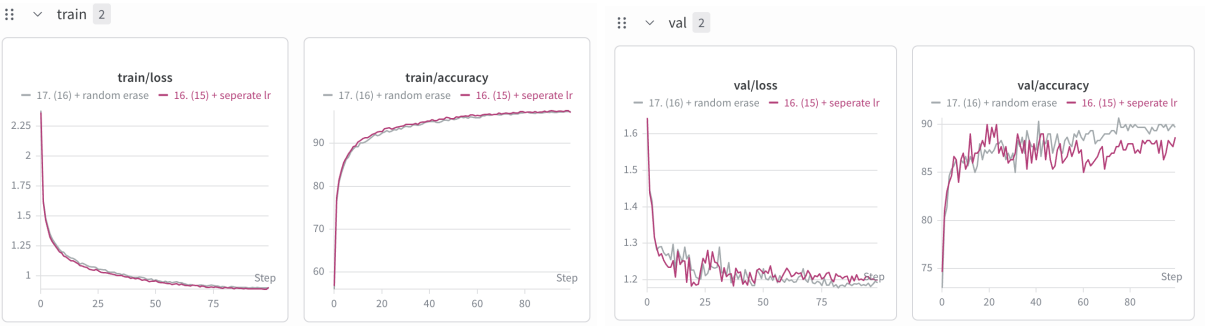

Figure 4: Training and validation loss curves of Separate LR model w/ and wo/ random erasing.

Table 3: Separate LR w/ and wo/ random erasing performance

Method	Test Score
Model from 3.1.2	0.94
Separate LR w/ random erasing	0.94
Separate LR wo/ random erasing	0.94

4 Additional Experiments

4.1 Experiment: Ensemble Learning - Bagging

Hypothesis & How this may work. Although a single ResNeXt-50 model already achieved strong performance, I think that applying Bagging could further reduce model variance and improve generalization on the test set. By training multiple models on different bootstrap samples, the ensemble should smooth out individual model biases and reduce the chance of overfitting to certain samples, potentially pushing performance beyond the $\sim$0.95 bottleneck.

Mechanism. I implemented Bagging by using RandomSampler and set replacement=True in the training DataLoader, and trained three ResNeXt-50 models with different seeds (42, 67, and 6767). For inference, I applied **soft voting** by averaging the per-class softmax probabilities from all three models to produce the final prediction.

Table 4: Test score comparison before and after Bagging.

Method	Test score
Single-model	0.95
Bagging ensemble	0.95

Results and Implications. Surprisingly, the Bagging ensemble yielded a test score of 0.95, showing no improvement over the single-model baseline. I came up with 2 possible reasons why it didn't work:

- **Diminishing Returns on Variance Reduction:** The primary theoretical advantage of Bagging is variance reduction. However, my current best model was already heavily regularized using data augmentations, weight decay, and other techniques. These techniques had already optimally constrained the model's variance, leaving almost no room for Bagging to provide further variance-reduction benefits.
- **Constrained Ensemble Diversity due to Transfer Learning:** Effective ensembling requires high diversity among base learners. Because we utilized a pre-trained ResNeXt-50 backbone with a strictly small layer-wise learning rate (10^{-4}), the feature extraction layers remained highly stable across all three models, regardless of the bootstrap sampling variations. Consequently, the bootstrap variations predominantly influenced only the newly initialized MLP classification head (trained with a larger LR of 10^{-3}). Ensembling models with near-identical feature extractors but slightly different linear heads lacked the architectural diversity required to push the accuracy beyond the 95% threshold.

5 References

The following references are provided for citation.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun. "Deep Residual Learning for Image Recognition." In *CVPR*, 2016.

[2] S. Xie, R. Girshick, P. Dollar, Z. Tu, and K. He. "Aggregated Residual Transformations for Deep Neural Networks." In *CVPR*, 2017.

[3] E. Boschkovskiy, A. G. Howard, and others. "TrivialAugment: Tuning-free Yet State-of-the-Art Data Augmentation." In *ICLR* (arXiv), 2021.

[4] Z. Zhong, S. Zheng, and others. "Random Erasing Data Augmentation." *arXiv preprint* arXiv:1708.04896, 2017.

[5] PyTorch. "ResNeXt-50-32x4d." https://pytorch.org/vision/stable/models/generated/torchvision.models.resnext50_32x4d.html